



Unity Catalog Metric Views



Goal: Turn data & context into insights

Lakehouse

Customer data



Website/
application/
logs



Marketing
campaigns



Financial data



Data Consumers



Dashboards



SQL access:
ODBC/JDBC/
SQL REST



Notebooks

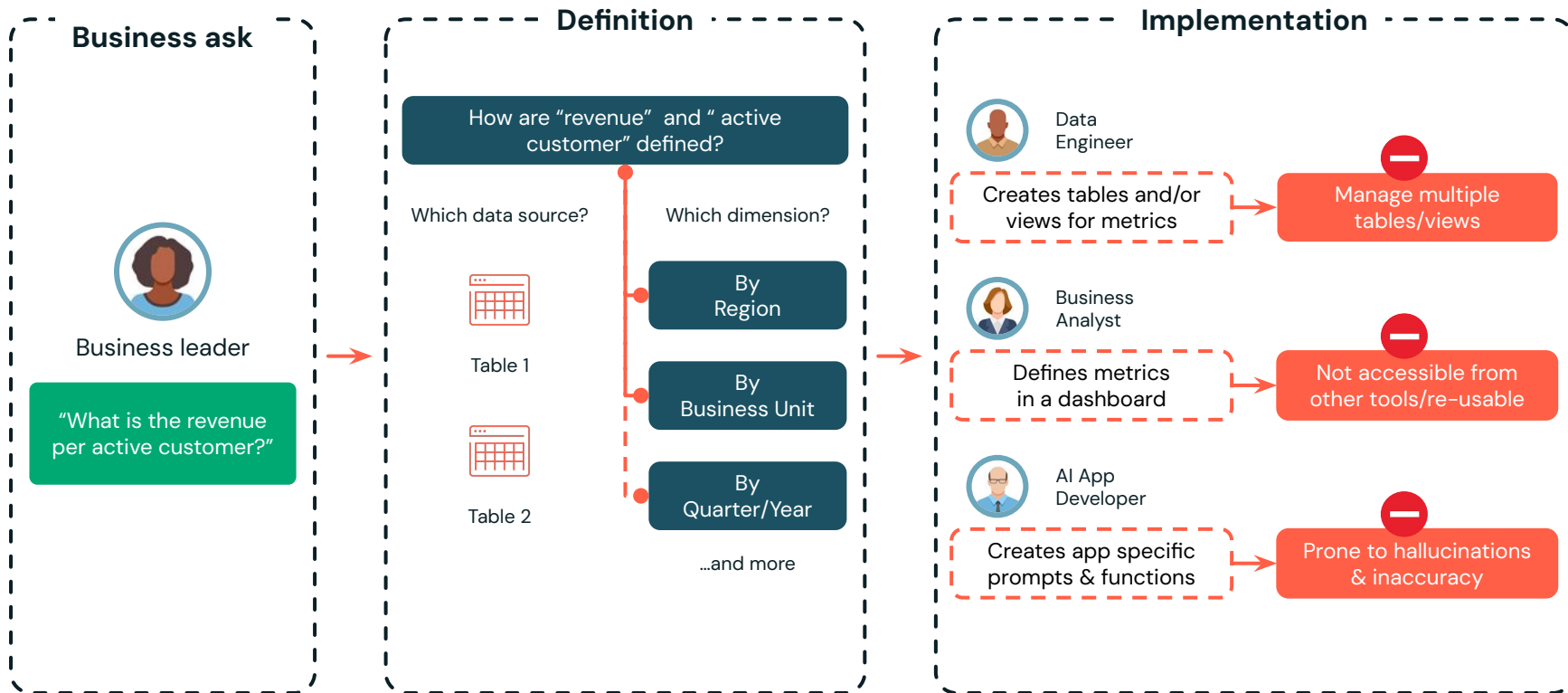


AI apps

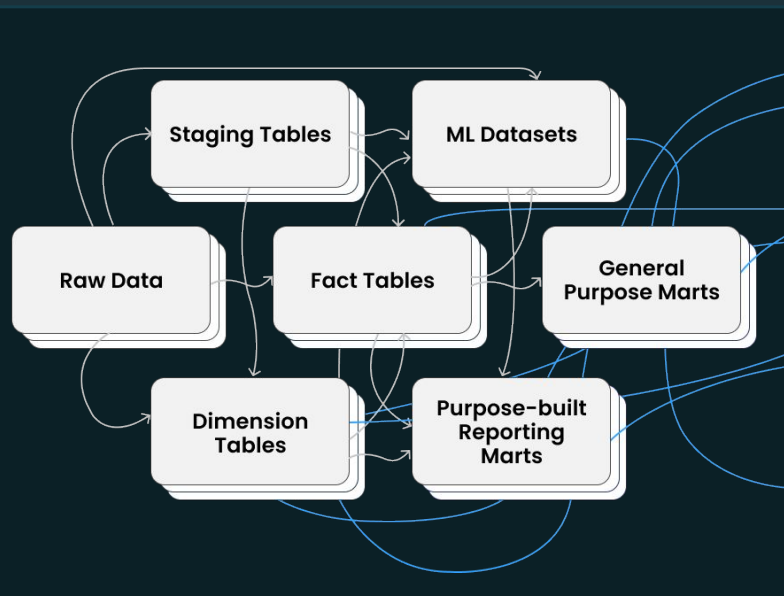
Need:
Enterprise-wide,
reliable and
consistent
business KPIs and
semantics



Hard in practice



Getting the right number is hard

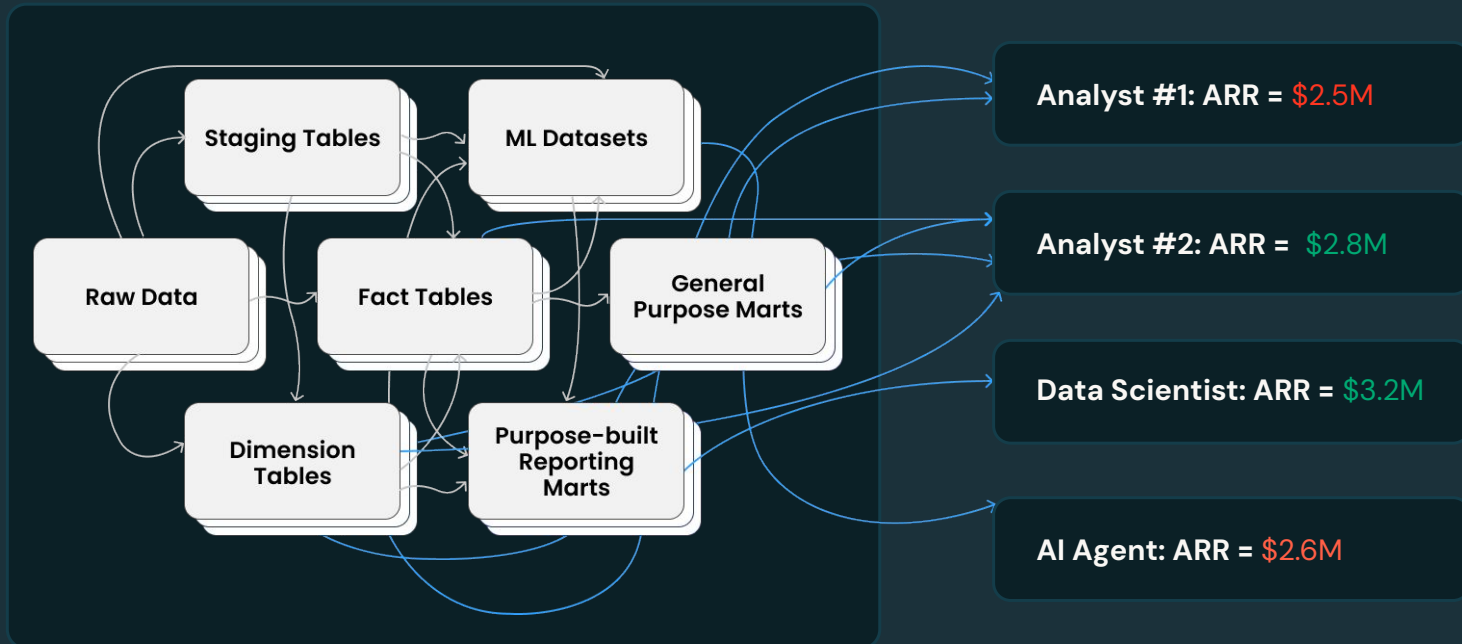


“What is our run-rate ARR?”

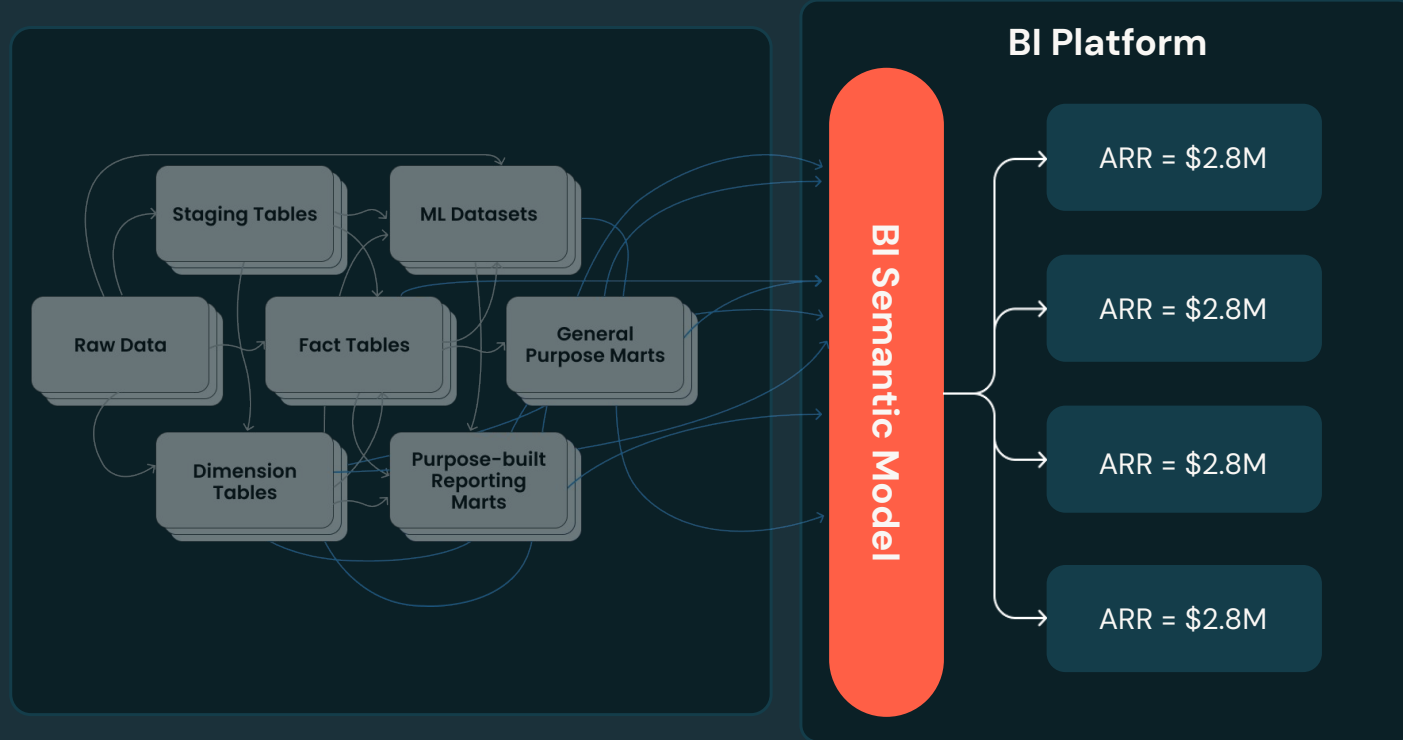
```
With dates AS (Select distinct date from
date_spline),
dailyData as (Select A.date,
sum(opportunities.amount) as bookings
From dates A
Left Join prod.oppo B
ON (B.date between date_sub(A.date, 6) and
A.date)
Group By A.date
Where opportunities.stage = 'Won'
)
Select date,
sum(bookings) OVER (ORDER BY date ASC ROWS 7 -
1 preceding) * 365 / 7 `AS Run-rate`
From dailyData
```



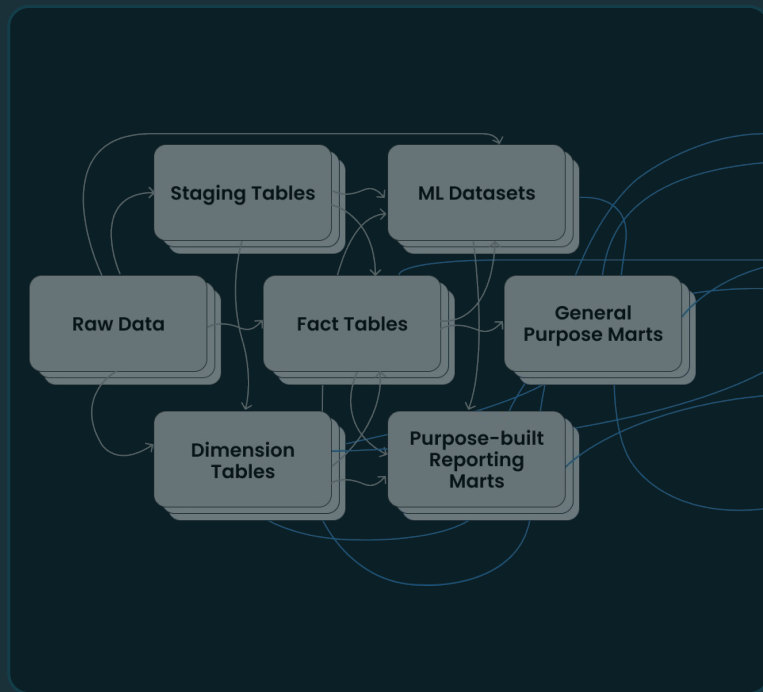
Getting it right *everywhere* is even harder



BI Semantic Layers aim to solve these problems



BI Semantic Layers are tool-specific



Semantic Layer #1	Power BI	ARR = \$2.5M
Semantic Layer #2	Tableau	ARR = \$2.4M
Semantic Layer #3	Looker	ARR = \$2.8M
Custom SQL	Jupyter, HEX, Plotly	ARR = \$3.1M

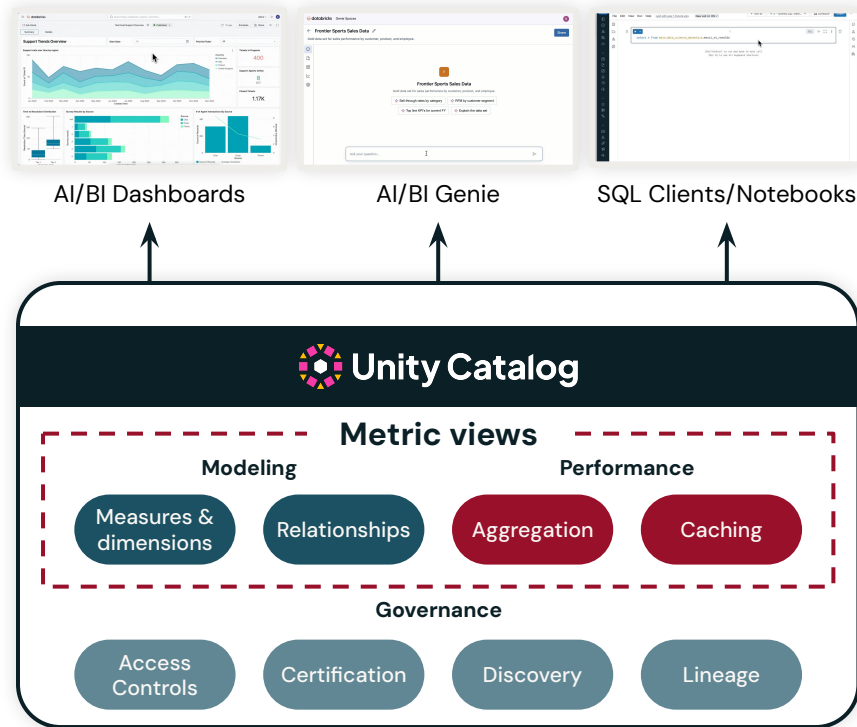


Introducing Unity Catalog Metric Views

Define your organization's business metrics in one place

Establish trust and reliability through centralized governance and certification

Access from AI/BI Dashboards, AI/BI Genie and SQL with the right semantics



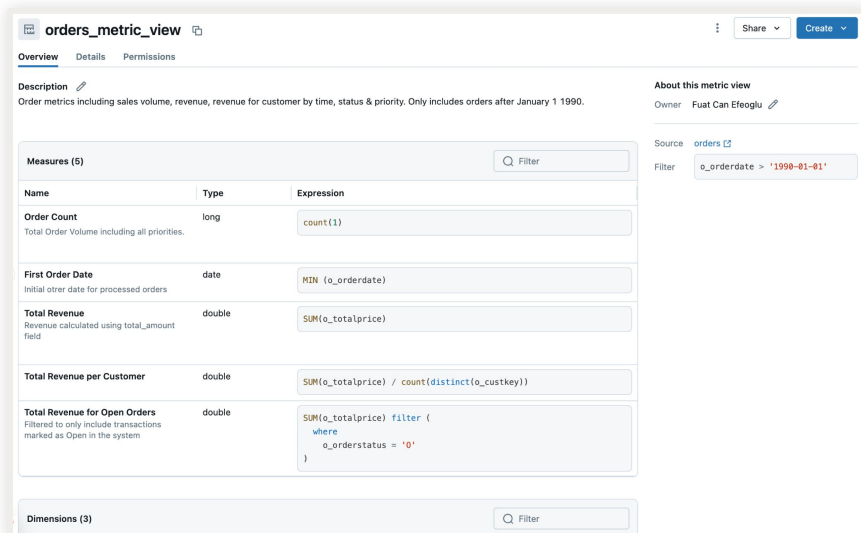
Reliable and consistent business metrics

Define your data model, relationships.
Build measures & dimensions.

Centrally govern with search, usage
insights, lineage & auditing built-in

Certify metrics for increased trust
and reliability

Materialize for performance and
pre-compute to reduce latency
for frequent queries



The screenshot displays a web interface for a metric view titled 'orders_metric_view'. It features a tabbed interface with 'Overview', 'Details', and 'Permissions'. The 'Overview' tab is active, showing a description of the metric view, a table of measures, and a section for dimensions. The 'About this metric view' section on the right indicates the owner is 'Fuat Can Efeoglu' and shows the source as 'orders' with a filter 'o_orderdate > '1998-01-01'.

Name	Type	Expression
Order Count	long	count(1)
First Order Date	date	MIN (o_orderdate)
Total Revenue	double	SUM(o_totalprice)
Total Revenue per Customer	double	SUM(o_totalprice) / count(distinct(o_custkey))
Total Revenue for Open Orders	double	SUM(o_totalprice) filter (where o_orderstatus = 'O')



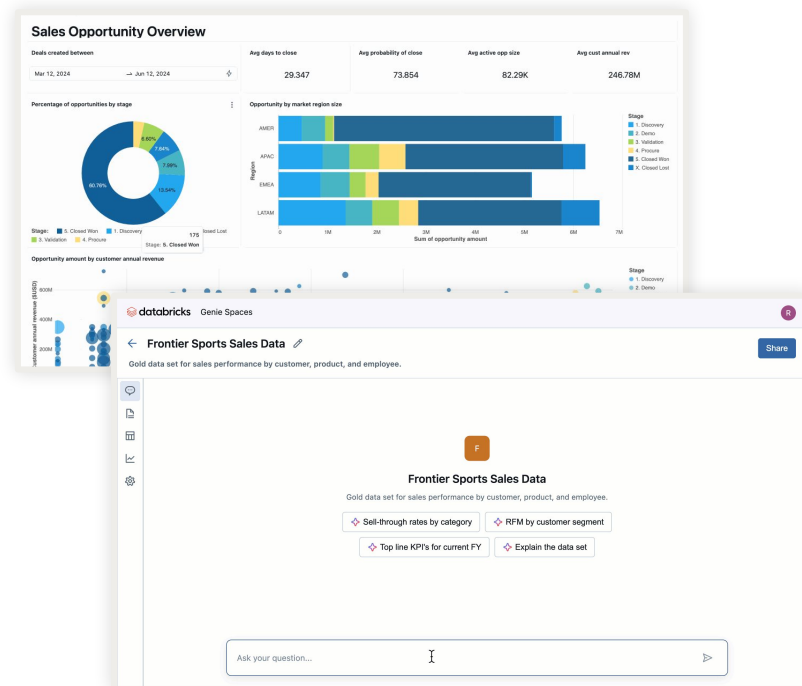
Accuracy across any Databricks tool, any workload

Integrated with **AI/BI Genie** spaces for **natural language querying**

Drag & drop **dashboarding** using **AI/BI Dashboards**

Query Metric Views using SQL from **notebooks, Gen AI or other applications**

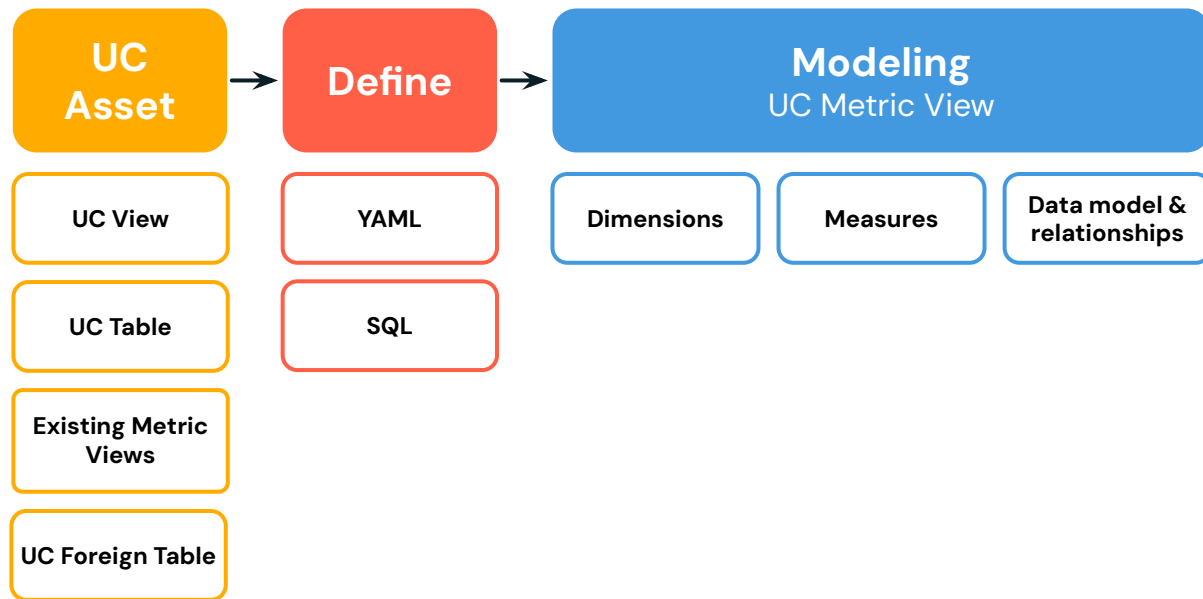
Query using various **clients via SQL**:
SQL REST API, ODBC, JDBC, Python,
GO, Node.js



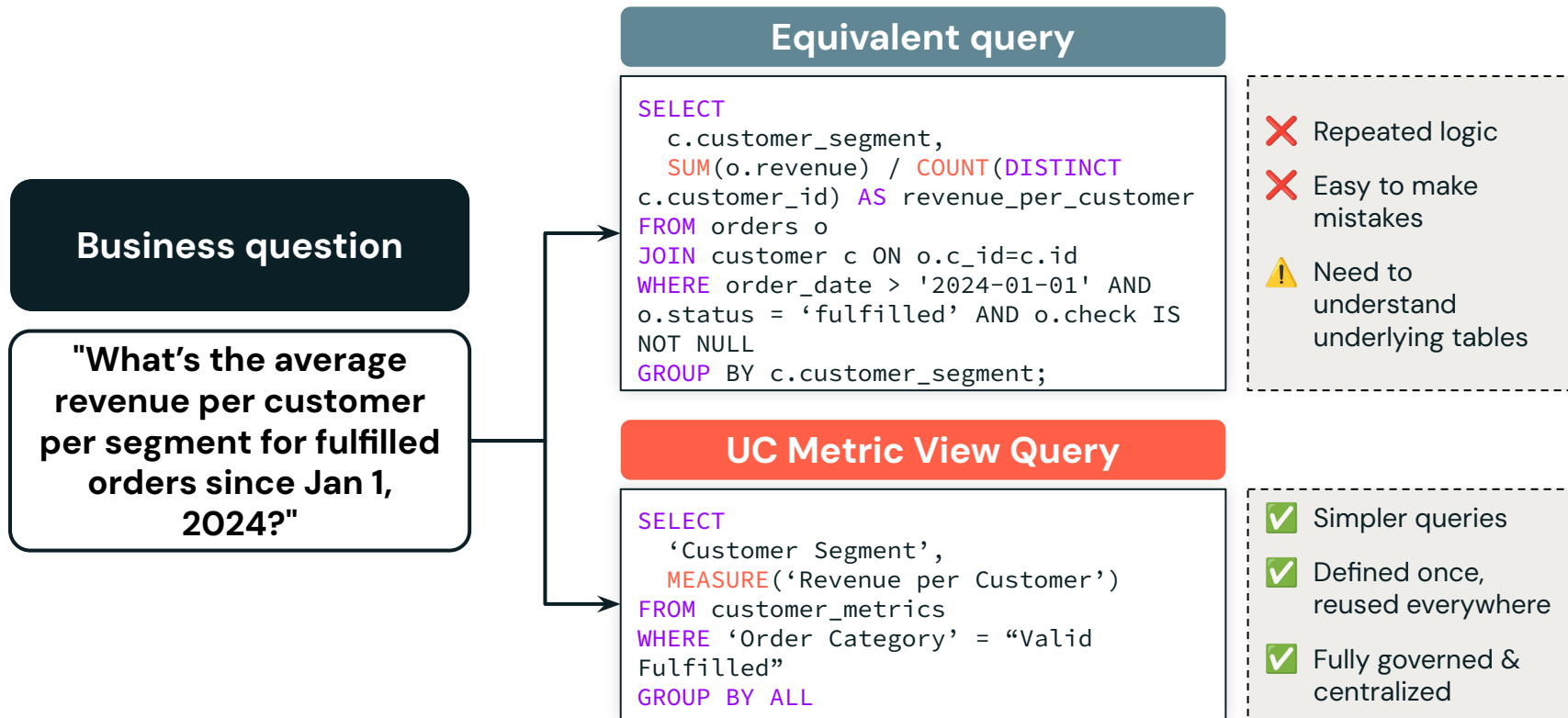
How it works



Define



Simplified, consistent insights



Defining Metric View

YAML to define a Metric View

A metric view is made up of:

- Source: Table or view containing raw data
 - ✓ e.g., sales_transactions
- Dimensions: Fields to filter or group by
 - ✓ e.g., region, product, channel
- Measures: Aggregations or KPIs
 - ✓ e.g., total_sales, avg_discount, order_count

One metric view = Many questions answered

- Mix & match measures and dimensions
- Supports slicing and filtering without redefining
- No need to create new assets for every slice or filter

Catalog Explorer > can > metric_view_testing >

Update definition for orders_metric_view

```
1  version: 1.0
2
3  source: samples.tpch.orders
4  filter: o_orderdate > '1990-01-01'
5
6  dimensions:
7    - name: Order Month
8      expr: date_trunc('MONTH', o_orderdate)
9      type: date
10
11    - name: Order Status
12      expr: >
13        case
14          when o_orderstatus = 'O' then 'Open'
15          when o_orderstatus = 'P' then 'Processing'
16          when o_orderstatus = 'F' then 'Fulfilled'
17        end
18      type: string
19
20    - name: Order Priority
21      expr: split(o_orderpriority, '-')[1]
22      type: string
23
24  measures:
25    - name: Order Count
26      expr: count(1)
27      type: bigint
28
29    - name: First Order Date
30      expr: MIN (o_orderdate)
31      type: date
32
33    - name: Total Revenue
34      expr: SUM(o_totalprice)
35      type: double
36
37    - name: Total Revenue per Customer
38      expr: SUM(o_totalprice) / count(distinct(o_custkey))
39      type: double
40
```



Define – Dimensions

dimensions:

```
- name: Order Date
  expr: orders.o_orderdate

- name: Order Status
  expr: >
    case
      when orders.o_orderstatus = 'O' then 'Open'
      when orders.o_orderstatus = 'P' then 'Processing'
      when orders.o_orderstatus = 'F' then 'Fulfilled'
    end

- name: Order Priority
  expr: >
    case
      when split(orders.o_orderpriority, '-') [0] = '1' then 'Urgent'
      when split(orders.o_orderpriority, '-') [0] = '2' then 'High'
      when split(orders.o_orderpriority, '-') [0] = '3' then 'Medium'
      when split(orders.o_orderpriority, '-') [0] = '4' then 'Low'
      else 'Unknown'
    end
```

Supports:

- Direct column references
- Any valid scalar SQL expression
- Scalar subqueries for advanced usage



Define – Dimensions

dimensions:

```
- name: Order Date
  expr: orders.o_orderdate

- name: Order Status
  expr: >
    case
      when orders.o_orderstatus = 'O' then 'Open'
      when orders.o_orderstatus = 'P' then 'Processing'
      when orders.o_orderstatus = 'F' then 'Fulfilled'
    end

- name: Order Priority
  expr: >
    case
      when split(orders.o_orderpriority, '-') [0] = '1' then 'Urgent'
      when split(orders.o_orderpriority, '-') [0] = '2' then 'High'
      when split(orders.o_orderpriority, '-') [0] = '3' then 'Medium'
      when split(orders.o_orderpriority, '-') [0] = '4' then 'Low'
      else 'Unknown'
    end
```

Supports:

- Direct column references
- Any valid scalar SQL expression
- Scalar subqueries for advanced usage



Define – Measures

```
measures:
- name: Total Revenue
  expr: SUM(l_extendedprice * (1 - l_discount))

- name: Order Count
  expr: COUNT(l_orderkey)

- name: Average Discount %
  expr: AVG(l_discount) * 100

- name: Average Order Value
  expr: MEASURE(`Total Revenue`) / MEASURE(`Order Count`)

- name: Fulfilled Revenue
  expr: MEASURE(`Total Revenue`)
    FILTER (WHERE orders.o_orderstatus = 'F')

- name: Trailing 7-Day Revenue
  expr: MEASURE(`Total Revenue`)
  window:
    - order: Order Date
      range: trailing 7 day
      semiadditive: last

- name: Cumulative Revenue
  expr: MEASURE(`Total Revenue`)
  window:
    - order: Order Date
      range: cumulative
      semiadditive: last
```

Supports:

- Any valid aggregate SQL expression
- Common windowing scenarios:
 - Trailing windows
 - Cumulative windows
 - Current / sub-total
- Composability: Use existing measures and/or dimensions to create new measures
- Scalar subqueries for advanced usage



Define – Measures

```
measures:
- name: Total Revenue
  expr: SUM(l_extendedprice * (1 - l_discount))

- name: Order Count
  expr: COUNT(l_orderkey)

- name: Average Discount %
  expr: AVG(l_discount) * 100

- name: Average Order Value
  expr: MEASURE(`Total Revenue`) / MEASURE(`Order Count`)

- name: Fulfilled Revenue
  expr: MEASURE(`Total Revenue`)
    FILTER (WHERE orders.o_orderstatus = 'F')

- name: Trailing 7-Day Revenue
  expr: MEASURE(`Total Revenue`)
  window:
    - order: Order Date
      range: trailing 7 day
      semiadditive: last

- name: Cumulative Revenue
  expr: MEASURE(`Total Revenue`)
  window:
    - order: Order Date
      range: cumulative
      semiadditive: last
```

Supports:

- Any valid aggregate SQL expression
- Common windowing scenarios:
 - Trailing windows
 - Cumulative windows
 - Current / sub-total
- Composability: Use existing measures and/or dimensions to create new measures
- Scalar subqueries for advanced usage



Querying Metric View

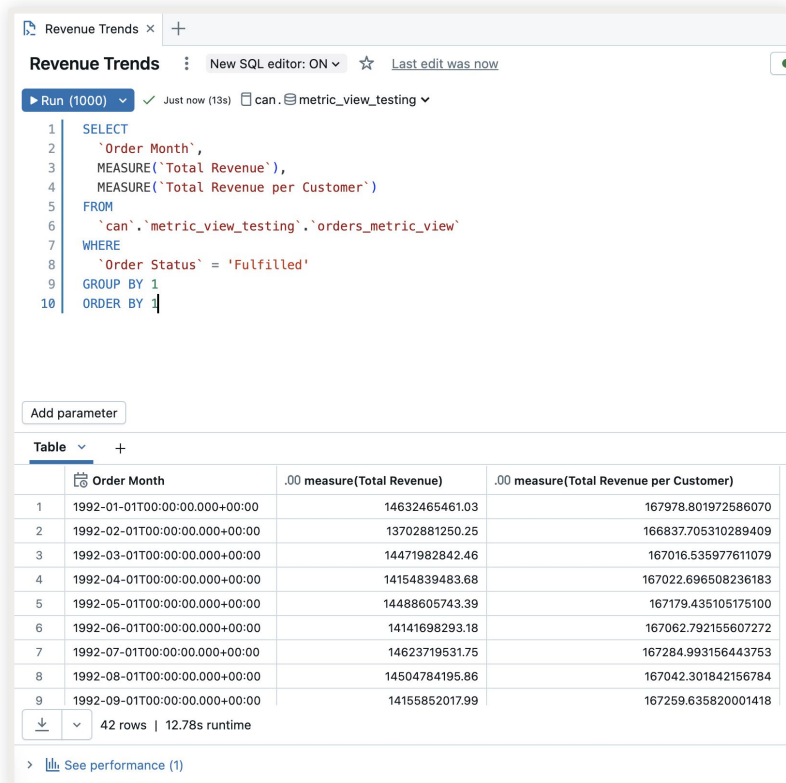
Query using MEASURE() Function

- Query business metrics directly using SQL
- No need to understand the underlying data model or schema layout

Metric Views automatically generate:

- Correct aggregations & expressions
- Required joins across tables
- Materialized views for performance optimization (if available and applicable)

**Just use SELECT MEASURE(...)—
let Metric Views handle
the complexity**



The screenshot shows a SQL query editor interface for a database. The query is titled "Revenue Trends" and is executed in the "New SQL editor: ON" mode. The query text is as follows:

```
1 SELECT
2   `Order Month`,
3   MEASURE(`Total Revenue`),
4   MEASURE(`Total Revenue per Customer`)
5 FROM
6   `can`.`metric_view_testing`.`orders_metric_view`
7 WHERE
8   `Order Status` = 'Fulfilled'
9 GROUP BY 1
10 ORDER BY 1
```

Below the query editor, there is a table of results. The table has four columns: "Order Month", ".00 measure(Total Revenue)", and ".00 measure(Total Revenue per Customer)". The table contains 9 rows of data, representing monthly revenue trends from 1992-01-01 to 1992-09-01. The table also shows a download icon, a dropdown menu, and a runtime time of 42 rows | 12.78s runtime.

	Order Month	.00 measure(Total Revenue)	.00 measure(Total Revenue per Customer)
1	1992-01-01T00:00:00.000+00:00	14632465461.03	167978.801972586070
2	1992-02-01T00:00:00.000+00:00	13702881250.25	166837.705310289409
3	1992-03-01T00:00:00.000+00:00	14471982842.46	167016.535977611079
4	1992-04-01T00:00:00.000+00:00	14154839483.68	167022.696508236183
5	1992-05-01T00:00:00.000+00:00	14488605743.39	167179.435105175100
6	1992-06-01T00:00:00.000+00:00	14141698293.18	167062.792155607272
7	1992-07-01T00:00:00.000+00:00	14623719531.75	167284.993156443753
8	1992-08-01T00:00:00.000+00:00	14504784195.86	167042.301842156784
9	1992-09-01T00:00:00.000+00:00	14155852017.99	167259.635820001418



Querying Metric View

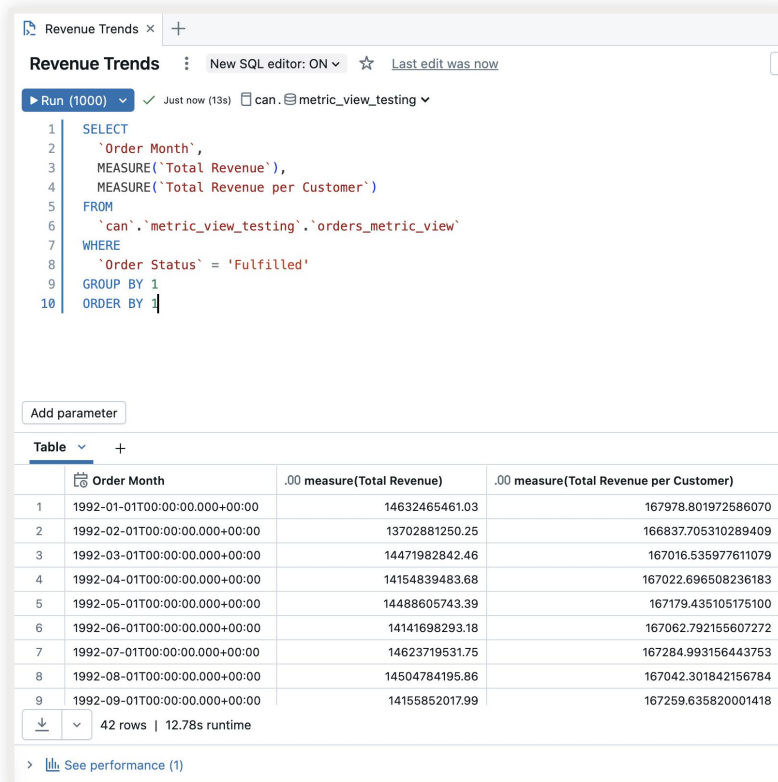
Query using MEASURE() Function

- Query business metrics directly using SQL
- No need to understand the underlying data model or schema layout

Metric Views automatically generate:

- Correct aggregations & expressions
- Required joins across tables
- Materialized views for performance optimization (if available and applicable)

**Just use SELECT MEASURE(...)—
let Metric Views handle
the complexity**



The screenshot shows a SQL query editor interface for a database. The query is titled "Revenue Trends" and is executed in the "New SQL editor: ON" mode. The query text is as follows:

```
1 SELECT
2   `Order Month`,
3   MEASURE(`Total Revenue`),
4   MEASURE(`Total Revenue per Customer`)
5 FROM
6   `can`.`metric_view_testing`.`orders_metric_view`
7 WHERE
8   `Order Status` = 'Fulfilled'
9 GROUP BY 1
10 ORDER BY 1
```

Below the query editor, there is a table of results. The table has four columns: "Order Month", ".00 measure(Total Revenue)", and ".00 measure(Total Revenue per Customer)". The table contains 9 rows of data, representing monthly revenue trends from 1992-01-01 to 1992-09-01. The table is titled "Table" and has a "42 rows | 12.78s runtime" status.

	Order Month	.00 measure(Total Revenue)	.00 measure(Total Revenue per Customer)
1	1992-01-01T00:00:00.000+00:00	14632465461.03	167978.801972586070
2	1992-02-01T00:00:00.000+00:00	13702881250.25	166837.705310289409
3	1992-03-01T00:00:00.000+00:00	14471982842.46	167016.535977611079
4	1992-04-01T00:00:00.000+00:00	14154839483.68	167022.696508236183
5	1992-05-01T00:00:00.000+00:00	14488605743.39	167179.435105175100
6	1992-06-01T00:00:00.000+00:00	14141698293.18	167062.792155607272
7	1992-07-01T00:00:00.000+00:00	14623719531.75	167284.993156443753
8	1992-08-01T00:00:00.000+00:00	14504784195.86	167042.301842156784
9	1992-09-01T00:00:00.000+00:00	14155852017.99	167259.635820001418



Views vs Metric Views | What's the difference?

Examples

Views or tables

```
CREATE VIEW view_1 AS
SELECT
  SUM(sale_amount) / COUNT(DISTINCT customer_id)
FROM orders
GROUP BY order_date
```

```
CREATE VIEW view_2 AS
SELECT
  SUM(sale_amount) / COUNT(DISTINCT customer_id)
FROM orders
GROUP BY region
```

```
CREATE VIEW view_3 AS
SELECT
  SUM(sale_amount) / COUNT(DISTINCT customer_id)
FROM orders
GROUP BY product_line
```

```
CREATE VIEW view_N AS
...
```

Metric view

Definition (*Single metric view instead of N views*)

order_metrics

Dimensions: order_date, region, product_line

Measures : SUM(sale_amount) / COUNT(DISTINCT customer_id)

Users pick GROUP BY at query time

```
SELECT
  Product_line,
  region,
  MEASURE (rev)
FROM order_metrics
WHERE order_date > '01-01-2024'
GROUP BY ALL
```



Views vs Metric Views | What's the difference?

Examples

Views or tables

```
CREATE VIEW view_1 AS
SELECT
  SUM(sale_amount) / COUNT(DISTINCT customer_id)
FROM orders
GROUP BY order_date
```

```
CREATE VIEW view_2 AS
SELECT
  SUM(sale_amount) / COUNT(DISTINCT customer_id)
FROM orders
GROUP BY region
```

```
CREATE VIEW view_3 AS
SELECT
  SUM(sale_amount) / COUNT(DISTINCT customer_id)
FROM orders
GROUP BY product_line
```

```
CREATE VIEW view_N AS
...
```

Metric view

Definition (*Single metric view instead of N views*)

order_metrics

Dimensions: order_date, region, product_line

Measures : SUM(sale_amount) / COUNT(DISTINCT customer_id)

Users pick GROUP BY at query time

```
SELECT
  product_line,
  MEASURE(rev)
FROM order_metrics
WHERE order_date > '01-01-2024'
GROUP BY 1
```



Unity Catalog Metrics Partners

Upcoming Integrations
(pending partner development)

